

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 - SORTING



NAMA : HILMAN ANIQ ALMUNAWA

NIM : 109082500027

KELAS : S1 IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Sorting atau pengurutan adalah proses menyusun sekumpulan data berdasarkan urutan tertentu, baik secara menaik (ascending) maupun menurun (descending). Dalam pemrograman, proses sorting sangat penting karena data yang sudah terurut akan jauh lebih mudah dicari, diolah, dan dianalisis.

Terdapat dua metode dasar pengurutan yang sering digunakan, yaitu Selection Sort dan Insertion Sort. Selection Sort bekerja dengan mencari nilai ekstrem (terkecil atau terbesar) dari sekumpulan data yang tersisa, kemudian menukarnya (swap) ke posisi yang seharusnya. Proses ini dilakukan berulang hingga hanya tersisa satu data dan seluruhnya terurut.

Sementara itu, Insertion Sort bekerja dengan cara yang berbeda, yaitu tidak mencari nilai ekstrem. Algoritma ini membandingkan data secara sekuensial, kemudian menyisipkan data tersebut ke posisi yang tepat pada bagian data yang sudah terurut agar keterurutannya tetap terjaga.

Dengan memahami Selection Sort dan Insertion Sort, proses pengolahan data dapat dilakukan dengan lebih terstruktur dan efisien. Kedua metode ini juga menjadi fondasi penting untuk mempelajari berbagai algoritma pengurutan yang lebih kompleks dalam pemrograman.

B. Guided

1. Program Selection Sort (Ascending)

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int

    for i = 0; i < len(arr)-1; i++ { // perulangan luar
        idx_min = i

        for j = i + 1; j < len(arr); j++ { // perulangan dalam
            if arr[j] < arr[idx_min] { // kondisi
                idx_min = j
            }
        }
    }
}
```

```

        if idx_min != i { // swap
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int
    var i int

    for i = 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }

    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")

    for i = 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }

    SelectionSortArray(&arrAngka)

    fmt.Println()
    fmt.Println("=== SETELAH SORTING ===")

    for i = 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}

```

Output :

```

PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\1.go"
Masukkan data angka indeks ke-0 : 8
Masukkan data angka indeks ke-1 : 3
Masukkan data angka indeks ke-2 : 4
Masukkan data angka indeks ke-3 : 1
Masukkan data angka indeks ke-4 : 6
Masukkan data angka indeks ke-5 : 9
Masukkan data angka indeks ke-6 : 5
Masukkan data angka indeks ke-7 : 4

=== SEBELUM SORTING ===
8 3 4 1 6 9 5 4
=== SETELAH SORTING ===
1 3 4 4 5 6 8 9
PS C:\Users\micro\Documents\codingan hilman\github>

```

2. Program Insertion Sort (Descending)

```
package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int

    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data : ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Printf("Masukkan data ke-%d : ", i)
        fmt.Scan(&data[i])
    }

    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")

    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)
    fmt.Println()
    fmt.Println("=== SETELAH SORTING DESCENDING ===")
}
```

```

    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\2.go"
Masukkan jumlah data : 7
Masukkan data ke-0 : 9
Masukkan data ke-1 : 1
Masukkan data ke-2 : 5
Masukkan data ke-3 : 7
Masukkan data ke-4 : 4
Masukkan data ke-5 : 3
Masukkan data ke-6 : 1

=== SEBELUM SORTING ===
9 1 5 7 4 3 1
=== SETELAH SORTING DESCENDING ===
9 7 5 4 3 1 1
PS C:\Users\micro\Documents\codingan hilman\github>

```

3. Pengurutan Array Descending (Insertion Sort)

```

package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

```

```

    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data:")
    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println()
    fmt.Println("=== SETELAH SORTING DESCENDING ===")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\3 .go"
Masukkan jumlah data: 7
Masukkan data:
9
2
5
7
8
1
3

=== SEBELUM SORTING ===
9 2 5 7 8 1 3
=== SETELAH SORTING DESCENDING ===
9 8 7 5 3 2 1
PS C:\Users\micro\Documents\codingan hilman\github>

```

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort. Masukan dimulai dengan sebuah integer n ($0 < n < 1000$) banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

Jawaban :

```
package main

import "fmt"

type arrInt [100000]int

func selectionSort(arr *arrInt, m int) {
    var idx_min, temp int
    for i := 0; i < m-1; i++ {
        idx_min = i
        for j := i + 1; j < m; j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        if idx_min != i {
            temp = arr[i]
            arr[i] = arr[idx_min]
            arr[idx_min] = temp
        }
    }
}

func main() {
    var n, m int
    var data arrInt

    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&m)
```

```

        for j := 0; j < m; j++ {
            fmt.Scan(&data[j])
        }

        selectionSort(&data, m)

        for j := 0; j < m; j++ {
            fmt.Print(data[j], " ")
        }
        fmt.Println()
    }
}

```

Output :

```

PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\
ded 1 .go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Users\micro\Documents\codingan hilman\github>

```

Penjelasan :

Program di atas berfungsi untuk mengurutkan nomor rumah kerabat Hercules dari yang terkecil ke terbesar menggunakan metode Selection Sort. Cara kerjanya, program pertama kali membaca jumlah daerah (n). Lalu program akan melooping sebanyak n kali untuk membaca jumlah rumah di tiap daerah (m) beserta nomor-nomor rumahnya ke dalam sebuah array. Setelah data masuk, saya memanggil fungsi selectionSort. Fungsi ini bekerja dengan cara mencari angka terkecil dari deretan data yang belum terurut, lalu menukarnya ke posisi paling depan. Proses ini diulang sampai semua nomor rumah terurut. Terakhir, array yang sudah rapi dicetak berderet ke samping.

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap

terurut mengecil. Format Masukan masih persis sama seperti sebelumnya. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

Jawaban :

```
package main

import "fmt"

type arrInt [100000]int

func selectionSortGanjilGenap(arr *arrInt, m int) {
    var ganjil, genap arrInt
    var nGanjil, nGenap int

    for i := 0; i < m; i++ {
        if arr[i]%2 != 0 {
            ganjil[nGanjil] = arr[i]
            nGanjil++
        } else {
            genap[nGenap] = arr[i]
            nGenap++
        }
    }

    for i := 0; i < nGanjil-1; i++ {
        idx_min := i
        for j := i + 1; j < nGanjil; j++ {
            if ganjil[j] < ganjil[idx_min] {
                idx_min = j
            }
        }
        ganjil[i], ganjil[idx_min] = ganjil[idx_min],
ganjil[i]
    }

    for i := 0; i < nGenap-1; i++ {
        idx_max := i
        for j := i + 1; j < nGenap; j++ {
            if genap[j] > genap[idx_max] {
                idx_max = j
            }
        }
        genap[i], genap[idx_max] = genap[idx_max], genap[i]
    }

    idx := 0
    for i := 0; i < nGanjil; i++ {
```

```

        arr[idx] = ganjil[i]
        idx++
    }
    for i := 0; i < nGenap; i++ {
        arr[idx] = genap[i]
        idx++
    }
}

func main() {
    var n, m int
    var data arrInt

    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&m)
        for j := 0; j < m; j++ {
            fmt.Scan(&data[j])
        }

        selectionSortGanjilGenap(&data, m)

        for j := 0; j < m; j++ {
            fmt.Print(data[j], " ")
        }
        fmt.Println()
    }
}

```

Output :

```

PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\
ded 2 .go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Users\micro\Documents\codingan hilman\github>

```

Penjelasan :

Program ini digunakan untuk mengurutkan nomor rumah kerabat Hercules dengan aturan nomor ganjil ditampilkan lebih dulu secara urut membesar, lalu nomor genap ditampilkan secara urut mengecil menggunakan metode Selection Sort. Cara kerjanya yaitu data dipisahkan dulu menjadi dua kelompok, yaitu ganjil dan genap. Setelah itu data ganjil diurutkan membesar dan data genap diurutkan mengecil, kemudian keduanya digabung kembali dengan urutan ganjil terlebih dahulu lalu genap. Untuk hasil yang berbeda dengan

contoh pada modul, kemungkinan terdapat kesalahan pada contoh keluarannya karena jika mengikuti deskripsi soal yang meminta ganjil terurut membesar dan genap terurut mengecil, maka hasil program yang dibuat sudah sesuai dengan ketentuan soal.

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya. Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array. Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban :

```
package main

import "fmt"

type arrInt [1000]int

func insertionSort(arr *arrInt, n int) {
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j--
        }
        arr[j] = temp
    }
}

func main() {
    var data arrInt
    var n, x int

    // Membaca data sampai bilangan negatif
    for {
        fmt.Scan(&x)
        if x < 0 {
```

```

        break
    }
    data[n] = x
    n++
}

insertionSort(&data, n)

for i := 0; i < n; i++ {
    fmt.Print(data[i], " ")
}
fmt.Println()

if n > 1 {
    jarak := data[1] - data[0]
    tetap := true
    for i := 1; i < n-1; i++ {
        if data[i+1]-data[i] != jarak {
            tetap = false
            break
        }
    }

    if tetap {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}
}

```

Output :

```

PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\
ded 3 .go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\
ded 3 .go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Users\micro\Documents\codingan hilman\github>

```

Penjelasan :

Program di atas berfungsi untuk membaca sekumpulan bilangan bulat, mengurutkannya menggunakan metode Insertion Sort secara membesar, lalu memeriksa apakah jarak antar bilangan yang sudah terurut bernilai tetap atau tidak. Cara kerjanya, program membaca data sampai ditemukan bilangan

negatif sebagai penanda akhir input. Data yang tersimpan kemudian diurutkan menggunakan Insertion Sort, setelah itu program menghitung selisih antar bilangan yang berurutan. Jika semua selisih bernilai sama maka program menampilkan "Data berjarak x", sedangkan jika ada selisih yang berbeda maka program menampilkan "Data berjarak tidak tetap".

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
Go
const nMax integer = 7919
type Buku = <
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating integer
>
type DaftarBuku array [1..nMax] of Buku
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari. Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir. Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku
```

```

func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
&pustaka[i].penulis, &pustaka[i].penerbit,
&pustaka[i].eksemplar, &pustaka[i].tahun,
&pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }
    terfavorit := pustaka[0]
    for i := 1; i < n; i++ {
        if pustaka[i].rating > terfavorit.rating {
            terfavorit = pustaka[i]
        }
    }
    fmt.Println("Buku Terfavorit:")
    fmt.Printf("%s, %s, %s, %d\n", terfavorit.judul,
terfavorit.penulis, terfavorit.penerbit, terfavorit.tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := pustaka[i]
        j := i
        // Sorting menurun (descending) berdasarkan
rating
        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j--
        }
        pustaka[j] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    fmt.Println("5 Judul Buku dengan Rating Tertinggi:")
    batas := 5
    if n < 5 {
        batas = n
    }
    for i := 0; i < batas; i++ {
        fmt.Printf("%d. %s\n", i+1, pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    kiri := 0

```

```

        kanan := n - 1
        ketemu := false

        for kiri <= kanan && !ketemu {
            tengah := (kiri + kanan) / 2
            if pustaka[tengah].rating == r {
                fmt.Println("Data Buku Ditemukan:")
                fmt.Printf("%s, %s, %s, %d, %d, %d\n",
pustaka[tengah].judul, pustaka[tengah].penulis,
pustaka[tengah].penerbit, pustaka[tengah].tahun,
pustaka[tengah].eksemplar, pustaka[tengah].rating)
                ketemu = true
            } else if pustaka[tengah].rating < r {
                kanan = tengah - 1 // Geser kanan karena
descending
            } else {
                kiri = tengah + 1 // Geser kiri karena
descending
            }
        }

        if !ketemu {
            fmt.Println("Tidak ada buku dengan rating seperti
itu")
        }
    }

    func main() {
        var pustaka DaftarBuku
        var n, ratingCari int

        fmt.Scan(&n)
        DaftarkanBuku(&pustaka, n)

        fmt.Println()
        CetakTerfavorit(pustaka, n)

        UrutBuku(&pustaka, n)

        fmt.Println()
        Cetak5Terbaru(pustaka, n)

        fmt.Scan(&ratingCari)
        fmt.Println()
        CariBuku(pustaka, n, ratingCari)
    }

```

Output :

```
PS C:\Users\micro\Documents\codingan hilman\github> go run "c:\Users\micro\Documents\codingan hilman\github\ded 4 .go"
3
B01 LaskarPelangi AndreaHirata Bentang 10 2005 95
B02 Bumi TereLiye Gramedia 8 2014 90
B03 Negeri5Menara AhmadFuadi Gramedia 12 2009 85

Buku Terfavorit:
LaskarPelangi, AndreaHirata, Bentang, 2005

5 Judul Buku dengan Rating Tertinggi:
1. LaskarPelangi
2. Bumi
3. Negeri5Menara
90

Data Buku Ditemukan:
Bumi, TereLiye, Gramedia, 2014, 8, 90
PS C:\Users\micro\Documents\codingan hilman\github> |
```

Penjelasan :

Program di atas berfungsi untuk mengelola data buku perpustakaan, menampilkan buku dengan rating tertinggi, mengurutkan data buku berdasarkan rating secara menurun menggunakan metode Insertion Sort, serta mencari buku berdasarkan rating menggunakan Binary Search. Cara kerjanya, program membaca sejumlah data buku yang terdiri dari id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Setelah seluruh data dibaca, program menampilkan buku terfavorit, kemudian mengurutkan data berdasarkan rating dari terbesar ke terkecil. Selanjutnya program menampilkan maksimal 5 judul buku dengan rating tertinggi dan melakukan pencarian buku sesuai rating yang dimasukkan pengguna. Jika ditemukan, data buku ditampilkan, sedangkan jika tidak ditemukan akan muncul pesan bahwa buku dengan rating tersebut tidak ada.

D. Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa sorting merupakan teknik pengurutan data yang digunakan untuk menyusun data agar lebih teratur dan mudah diolah. Pada praktikum ini digunakan dua metode pengurutan yaitu Selection Sort dan Insertion Sort. Selection Sort bekerja dengan mencari nilai terkecil atau terbesar dari data yang belum terurut kemudian menempatkannya pada posisi yang sesuai, sedangkan Insertion Sort bekerja dengan menyisipkan setiap data ke posisi yang tepat pada bagian data yang sudah terurut.

Melalui praktikum ini juga dapat dipahami bahwa proses sorting sangat berguna dalam berbagai pengolahan data, seperti mengurutkan nomor rumah, memisahkan bilangan ganjil dan genap sesuai aturan tertentu, memeriksa keteraturan jarak antar data, serta mengurutkan data buku berdasarkan rating. Selain itu, penggunaan array, struct, dan subprogram membantu program menjadi lebih terstruktur, mudah dipahami, dan lebih efisien dalam pengolahan data.

Dengan memahami konsep Selection Sort dan Insertion Sort, kemampuan dalam membuat program pengurutan data menjadi lebih baik serta dapat diterapkan pada berbagai kasus pemrograman lainnya yang memerlukan pengolahan dan penyajian data secara terurut.